

ARK Platform The Cognitive Engineering OS

A unified product, architecture, and roadmap brief for the ARK Platform — JST career intelligence, the SPHINX × Matrix marketplace, the CCGE Context Craft arena, and the Bonsai onboarding stack — under the ATANDA Studio banner.

We don't build AI agents. We engineer the DNA that governs them — powered by your cognition, owned by you.



Table of Contents

Front matter

Executive summary
.....

Vision & product positioning
.....

Personas & user benefits
.....

Specification

Canonical score glossary
.....

System architecture
.....

Data model overview
.....

Feature surface catalog

Identity (ARK / JST / CCMI / LHCS)
.....

Resume analyzer
.....

CCGE Arena
.....

SPHINX × Matrix marketplace
.....

Matrix Forge Lab
.....

Bonsai seller onboarding
.....

Cohorts & institutional
.....

GUIN+ identity
.....

Billing & subscription
.....

AI surfaces (Claude)
.....

GDPR / CCPA
.....

Admin & seed
.....

Engineering depth

Phase J ARK identity flywheel

Trust, security & privacy

Roadmap delta

Comparative analysis

Cross-PDD matrix

Per-PDD review (8 prior documents)

Convergence narrative

Appendices

A · Full HTTP route table

B · Score constants & weights

C · Bonsai 18-stage map

1. Executive Summary

ARK is a full-stack cognitive engineering platform that turns the way a person thinks about their work into a measurable, marketable, governable asset. It does this by composing four engines into a single product loop:

1. **A career intelligence engine** — the JST (Jobs · Skills · Talent) scorer that reads a resume and returns a tri-dimensional score out of 300, a 12-vector transferability map, and an AI-vulnerability profile.
2. **A context engineering engine** — the CCGE (Context Craft Game Engine) that grades the way users prompt and reason, returning a CCMI (Context Craft Mastery Index) score out of 300 and a Knowledge·Clarity·Specificity·Efficiency (KCSE) breakdown per session.
3. **A creator economy** — the SPHINX × Matrix marketplace where certified users publish Super Prompt Cards (SPCs), buyers stack them into synergistic decks, and the platform measures the social and economic ripple via a Roundtable awareness graph.
4. **An onboarding spine** — the 18-stage Bonsai walkthrough that turns a new seller from "I have an idea" into "my first publishable card has cleared HIVE Gold-80 and is live on Matrix."

The product is ARK identity-first. Every action — uploading a CV, finishing a CCGE session, publishing an SPC — feeds into a single canonical score ($ARK = JST + CCMI$, capped at 600) that streams live to the user's dashboard over SSE. The platform's positioning, captured in the v3 manifesto, is deliberately contrarian:

We don't build AI agents. We engineer the DNA that governs them — powered by your cognition, owned by you.

That sentence is the product. ARK does not ship a fleet of chatbots. It ships the **cognitive substrate** that makes any agent — human, AI, or institutional — measurable, accountable, and tradeable.

What v3 captures that prior PDDs did not

Eight prior PDDs (Feb–May 2026) progressively defined fragments of this product: the JST engine, the Forge Promptware methodology, the Junglenomics card economy, the GUIN+ identity layer, the Onecraft compression discipline, the ATANDA Command Centre operational model, and the Matrix Marketplace economic layer. v3 is the first PDD that:

- Treats **ARK identity** as the canonical product surface (single score, single writer, single SSE stream).
- Treats **Bonsai onboarding** as a first-class seller funnel rather than a marketing afterthought.
- Reconciles the manifesto stance (cognitive DNA, not agents) with the engineering reality (single-writer flywheel, capped deltas, atomic transactions).
- Includes the **ATANDA Studio** parent brand explicitly, both at the product surface (footer, sidebar) and at the corporate identity level (this document).

2. Vision & Product Positioning

2.1 Market thesis

The agent economy is real, but the value capture inside it is mis-positioned. Most platforms are racing to ship agents — interchangeable software substitutes for human work. ARK takes the inverse position: the durable, defensible asset is the **cognitive instruction layer** that governs agents. That instruction layer has three properties markets currently fail to price:

- **Provenance** — who authored the prompt-craft and what's their certified mastery level?
- **Synergy** — which prompts compose well together, and which destroy each other's signal?
- **Governance** — what guardrails (constraints, output schema, failure modes) are baked in, and have they cleared an independent quality gate (HIVE)?

ARK builds the registry, the scorer, and the marketplace for that layer.

2.2 Three audiences, one product

ARK is a single product but reads correctly to three audiences:

- **Individual professionals**: "Know your worth. Know your risk. Know your next move." The JST + vulnerability + pivot stack answers all three in under 60 seconds from a CV upload.
- **Creators**: "Turn your reasoning into a tradeable instrument." The CCGE → CCMI → CC_400 cert → SPHINX publish funnel converts personal prompt-craft into priced inventory.
- **Institutions**: "Govern your cohort's cognitive engineering." The instructor-gated cohort manager (assignments, grades CSV, comparison views) brings ARK into academies, bootcamps, and corporate L&D.

2.3 Why now

Three structural conditions converge in 2026 to make ARK timely:

1. **AI vulnerability is no longer abstract.** Average automation risk by occupation is published, debated, and increasingly priced into hiring decisions. Workers need a

personal vulnerability number, not an industry average.

2. **Prompt-craft has stabilized into a discipline.** It has frameworks (Context Engineering Pillars), benchmarks (JCSE), and certifications (CC_xxx ladder). It's now teachable, measurable, and gradeable — the prerequisites for a marketplace.
3. **The agent economy has begun.** Builders are paying for instructions, not just for inference. SPHINX is the asset class that emerges from that demand.

3. Personas & User Benefits

3.1 The Job-Seeking Professional ("Sarah")

Benefit: "I uploaded my CV and in 60 seconds I knew my market value, my AI risk, and the two skills that would pay off most. I built a 90-day plan from the same screen."

The flow: `/upload` → analyzer pipeline (5 phases) → `/dashboard` with JST gauge + radar, vulnerability meter, transferability radar, three pivot opportunities, three upskilling plans, and a print-ready Executive Summary at `/report` .

3.2 The Prompt-Craft Creator ("Marcus")

Benefit: "My prompt is no longer a screenshot in a Slack DM. It's a card with provenance, a HIVE Gold rating, and a price. People stack it into their decks and I see the synergy."

The flow: `/play` (CCGE Arena raises CCMI) → CC_400 cert achieved → `/marketplace/forgelab` (upload .docx, pre-check HIVE) → `/marketplace/publish` (price, taxonomy, description) → SPHINX live listing → Roundtable surfaces it via complementary-pair graph.

3.3 The Educator / Institution ("Dr. Patel")

Benefit: "I run a cohort of 40 students. I can assign Context Craft tasks, see the grade distribution, compare cohorts, and export a CSV the registrar accepts."

The flow: `/school` instructor dashboard → create cohort → bulk-add students by email → create assignments → grades view + `/api/cohorts/:id/grades.csv` (RFC-4180).

3.4 The Enterprise Buyer ("Tomás")

Benefit: "I see departmental vulnerability and JST distributions. I know where to invest in upskilling without surveying every employee individually."

The flow: `/enterprise` workforce intelligence dashboard with department heatmap, vulnerability pie, JST trend.

3.5 The Investor / Partner ("Ava")

Benefit: "Five minutes on the demo and I understand the product loop, the asset class, and the moat."

The flow: `/demo` (no login required) → static Sarah Chen persona renders the full JST/radar/vulnerability/pivot/upskilling stack → CTA returns to login or to the Bonsai onboarding teaser.

4. Canonical Score Glossary

All quantitative terms align to the Junglenomics FORGE Institute registries (General Technical Terms Registry v1.0 + Master SPC & Platform Registry v1.0, May 2026). Single source of truth: `shared/schema.ts::SCORE_GLOSSARY`.

TERM	RANGE	FORMULA / DEFINITION	CANON
ARK	0-600	<code>JST + CCMI</code>	ARK MAXIMUS ULTRA SI
JST	0-300	<code>(Jobs·0.30 + Skills·0.40 + Talent·0.30) × 3</code>	Jobs-Skills-Talent Career Assessment Agent
CCMI	0-300	weighted P1-P7 sum × 3	Context Craft Mastery Index
JCSE	0-50	composite session/agent quality	Composite AI Agent Quality Score
KCSE	0-100 each dim	Knowledge·0.30 + Clarity·0.30 + Specificity·0.20 + Efficiency·0.20	ARK-internal CCGE in-game rubric
HIVE	0-100	14-dim certification framework	HIVE (canon Term 8)
CC_xxx	ladder	<code>CC_100</code> =Foundational, <code>CC_200</code> =Bronze, <code>CC_300</code> =Silver, <code>CC_400</code> =Gold, <code>CC_500</code> =Platinum	ARK-internal cert ladder
Knight ranks	ladder	Squire / Knight / Paladin / Champion / Legend	GUIN+ contributor gamification

Tier breakpoints

- JCSE: Bronze 30 / Silver 36 / Gold 43 / Platinum 48 (constant `JCSE_TIER_THRESHOLDS`)
- HIVE: Bronze 60 / Silver 70 / Gold 80 / Platinum 90 — **publish gate is HIVE 80 (Gold)** to match the CC_400 user-cert floor.

Why two rubrics? KCSE grades a player's *card hand* inside the CCGE game; the canon JCSE rubric (Context Engineering Pillar 40% + Synergy 30% + Compression 20% + Semantic Preservation 10%) grades a *finished prompt artifact*. The two intentionally diverge because they measure different objects.

5. System Architecture

5.1 Stack

- **Frontend**: React + Vite + TailwindCSS + Recharts + Framer Motion. Routing via `wouter`. State via TanStack Query.
- **Backend**: Express.js on port 5000. Serves the JSON API, the SSE stream, and the Vite dev server in the same process. TypeScript-end-to-end via `tsx`.
- **Database**: PostgreSQL with Drizzle ORM. PG-backed session store via `connect-pg-simple`.
- **AI**: Anthropic Claude via the `javascript_anthropic_ai_integrations` blueprint. Haiku for KCSE; Sonnet for narrative + scenario generation.

5.2 Auth & sessions

- Server-side sessions only. Cookie `ark.sid`, `httpOnly`, `sameSite=lax`, 14-day rolling expiry. `SESSION_SECRET` required in production.
- Bcrypt-hashed passwords (cost 10) at the storage boundary. Transparent legacy-plaintext rehash on first successful login.
- Two middlewares: `requireAuth` (session-derived identity) and `requireSelf(:param)` (URL param must equal session userId). All mutations derive actor from `req.session.userId`, never from request body or URL path.
- Instructor-only routes guarded by `requireInstructor`.

5.3 Security headers & limits

- `helmet()` with production CSP, HSTS, `X-Frame-Options: SAMEORIGIN`, COOP, CORP, `Referrer-Policy: no-referrer`.
- JSON body limit: 1 MB. Resume upload: 10 MB. Forge Lab .docx upload: 5 MB.
- Rate limits: 240 req/min global, 20 failed auths / 15 min per IP.

5.4 Live updates: SSE orchestrator

The orchestrator (`server/orchestrator.ts`) is a typed event bus. Flywheel events (`assessment.completed` , `ccge.session.finalized` , `sphinx.purchase` , `ark.identity`) fan out to per-user SSE channels at `/api/ark-score/stream` .

The dashboard widget (`useArkStream`) snapshot-merges the live feed so the ARK identity card updates without a refresh.

5.5 AI integration

- All Claude calls go through `server/ai/client.ts` , gated by a per-user token-budget check (`server/ai/usage.ts`).
- KCSE judgments are cached (`server/ai/cache.ts`) to bound cost per session.
- Scenario generation is admin-gated.

5.6 Deployment

Target: **Replit autoscale**. Production refuses to start without `SESSION_SECRET` . `DATABASE_URL` and Anthropic credentials are auto-provisioned by the Replit integration system.

6. Data Model Overview

The schema (`shared/schema.ts`) lives at 34 Drizzle tables, grouped:

Identity & auth (3): `users` , `assessments` , `notifications` .

Career intelligence outputs (4): `upskilling_plans` , `pivot_opportunities` , `transferability_vectors` , `jnomics_cards` .

Reference data (2): `departments` , `ccge_cards` .

CCGE (2): `ccge_scenarios` , `game_sessions` .

SPHINX × Matrix marketplace (8): `spc_listings` , `spc_purchases` , `endorsements` , `user_credits` , `card_synergies` , `complementary_pairs` , `roundtable_state` , `synthesis_sessions` + `synthesis_creators_split` .

Billing (3): `checkoutSessions` , `billing_events` , `platform_credit_ledger` .

ARK identity flywheel (4): `ark_events` , `ccmi_pillar_scores` , `ark_score_history` , `lhcs_signals` .

AI accounting (2): `ai_usage` , `ai_cache` .

Institutional (3): `cohorts` , `cohort_memberships` , `cohort_assignments` .

QA & onboarding (2): `test_results` , `bonsai_progress` .

Each table has a `createInsertSchema` Zod insert schema; mutation routes validate against it before hitting storage.

7. Feature Surface Catalog

Each surface below names the **user benefit** (non-technical), its **capabilities**, its **endpoints**, and the **key files** that implement it.

7.1 Identity (ARK / JST / CCMI / LHCS)

User benefit: A single, live score that tells you how marketable you are and how good a thinker you are — updated the moment you do anything on the platform.

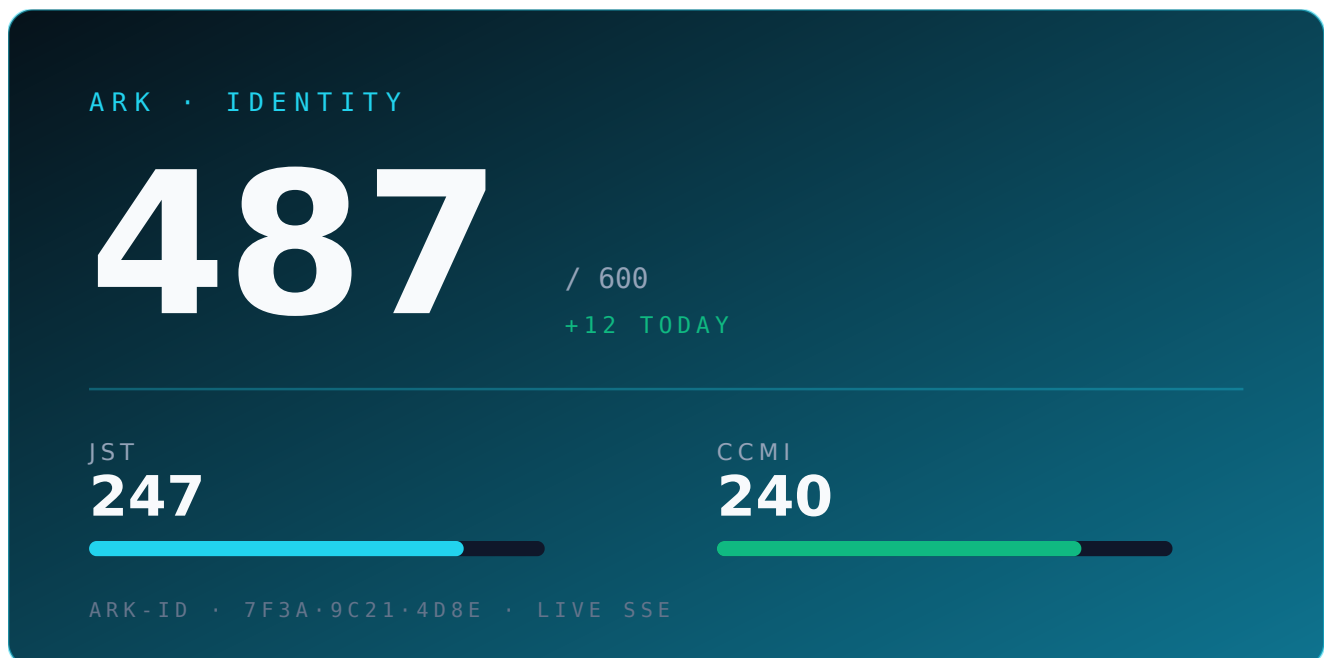


Figure 7.1b — ARK Identity Card: canonical product surface, single score = JST + CCMI, streamed live over SSE.

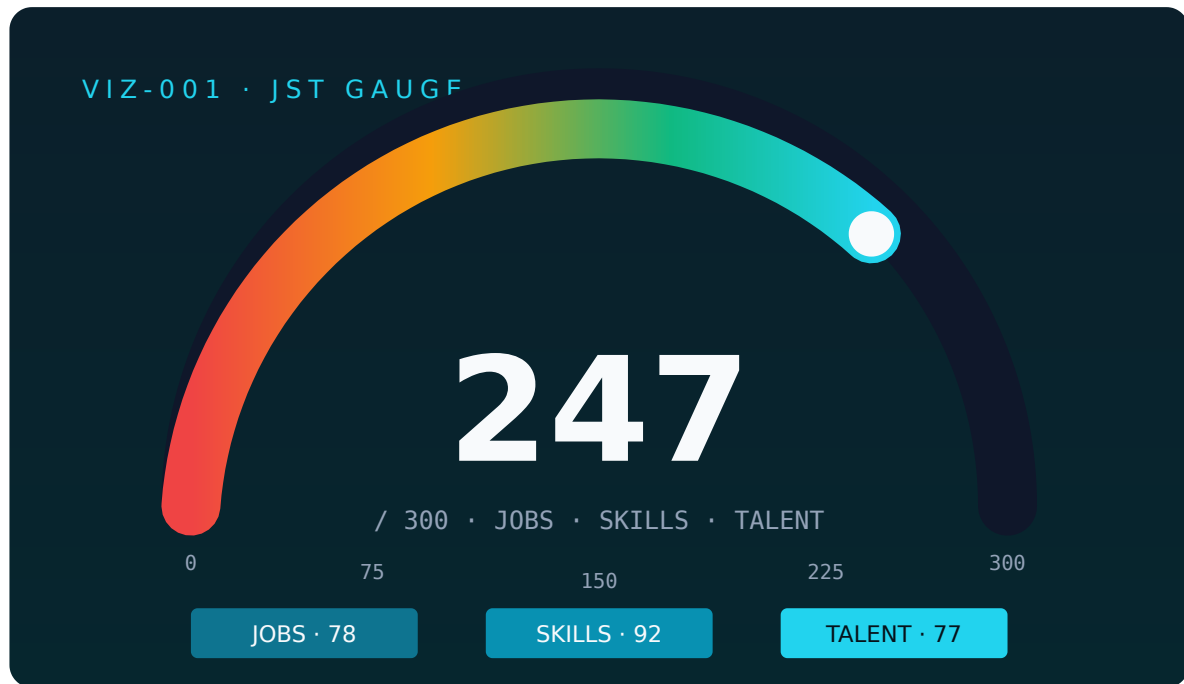


Figure 7.1a – JST Gauge (VIZ-001): tri-dimensional 0–300 readout with sub-dimension chips.

- Single identity formula: $\text{ARK} = \text{JST} + \text{CCMI}$, capped at 600.
- Single writer (`server/arkRecalc.ts`) updates `users`, `ccmi_pillar_scores`, `lhcs_signals`, and `ark_score_history` atomically.
- Flywheel caps: CCGE awards at most +15 ARK per day; SPHINX awards at most +20 ARK per rolling 30 days. Any rounding overshoot is hard-clamped off the CCMI side before persistence (`applyCaps` returns `intendedCap` ; recalc enforces $\text{delta} \leq \text{intendedCap}$).
- LHCS composite is a PDD-exact weighted blend: $\text{round}(0.35 \cdot \text{CPR} + 0.35 \cdot \text{MPS} + 0.30 \cdot \text{LCIS})$. Status (ACTIVE / DEVELOPING / BASELINE) is the threshold band of the `composite`, not a roll-up of the three traffic lights.
- Endpoints: `GET /api/ark/identity`, `POST /api/ark/recalc`, `GET /api/ark/flywheel-cta`, `GET /api/ark/history`, `GET /api/ark/lhcs`, `GET /api/ark-score/stream` (SSE), `GET /api/ark-score/events` (fallback).
- Files: `server/scoringEngine.ts`, `server/arkRecalc.ts`, `server/orchestrator.ts`, `client/src/components/dashboard/ArkIdentityCard.tsx`, `client/src/lib/useArkStream.ts`.

7.2 Resume Analyzer

User benefit: Upload your CV once; get a market score, a personal AI-risk profile, and an action plan you can hand to a recruiter — in under 60 seconds.

- Keyword scoring across 6 categories (technical, leadership, analytical, communication, innovation, AI-adjacent), each with weighted lists. Output feeds JST sub-dimensions.
- AI vulnerability: 14 regex task patterns → automation-risk average, adjusted by AI/leadership scores. Maps to a 0–4 vulnerability level rendered as a five-zone meter.
- **Archetype handicap system** — three weighted signals classify the resume as **Architect**, **Orchestrator**, or **Conductor**:
 - Job titles from `JST_ARCHETYPE_MAP` (228+ titles) — 40%
 - Skill-category weights — 35%
 - Matched FORGE card classifications — 25%
- **Context Craft handicap** — a multiplicative JST modifier (NONE 0.5× → CC_500 1.5×). Raw pre-multiplier values preserved in `jstRawTotal / jstRawJobs / jstRawSkills / jstRawTalent` for audit.
- Generates 12 transferability vectors, 3 pivot opportunities, 3 upskilling plans.
- Endpoints: `POST /api/resume/upload` (multipart, PDF or text), `POST /api/assessments`, `POST /api/assessment/text`, `GET /api/assessments/user/:userId/latest`.
- Files: `server/resumeAnalyzer.ts`, `client/src/pages/upload.tsx`.

7.3 CCGE Arena

User benefit: A card game that teaches you to think more clearly — and grades you while you play, so your dashboard score rises in real time.

- Single-player card game: deal 5 cards, write a hand for a tiered scenario, submit, receive KCSE-rubric judgment.
- Tiers: Bronze / Silver / Gold scenarios. Each finished session writes a `game_sessions` row and triggers ARK recalc.
- Optional Claude (Haiku) judging via `useClaude: true` on finalize — otherwise deterministic local scoring.

- Endpoints: `GET /api/ccge/cards` , `GET /api/ccge/scenarios` , `GET /api/ccge/sessions/:id` , `POST /api/ccge/sessions` , `POST /api/ccge/sessions/:id/finish` , `POST /api/ccge/scenarios/custom` .
- Files: `server/ccge.ts` , `server/ai/kcse.ts` , `client/src/pages/play.tsx` .

7.4 SPHINX × Matrix Marketplace

User benefit: Buy and sell the actual prompts that drive AI behavior — with quality ratings, synergy hints, and provenance baked in.

ATLAS_ULTRA_SI · v1.0
HIVE · GOLD 84

SUPER PROMPT CARD · P3 RECURSIVE SYSTEMS

A senior strategist's framework for evaluating multi-agent orchestration under cost ceilings.

CC_500

SYNERGY 92

COMPRESS 88

CREATOR · @velourian · CC_500 PLATINUM

PAIRS WITH · BUGMXT_SI · SPARTAN_MAX

PURCHASE · 45 CR

45 CR

Figure 7.4 – SPHINX × Matrix listing card: HIVE-gated, synergy-scored, creator-attributed, transactional purchase.

- Listings (`spc_listings`) gated by `SPC_MIN_CERT_TO_PUBLISH = CC_400` and a HIVE pre-check that returns 14-dimensional scores and a Gold/Silver/Bronze recommendation.
- Transactional purchase (`server/sphinx.ts::executePurchase`): debits credits, transfers listing-body access, awards ARK delta, records ledger entry, all in one DB transaction with row-level locking.
- Roundtable awareness graph** — global complementary-pair scoring across all listings, recomputed on writes, exposed at `GET /api/sphinx/roundtable`

and per-listing at `GET /api/sphinx/listings/:id/complementary` .

- **Synthesis sessions** — multi-creator collaborative composition; revenue split is recorded in `synthesis_creators_split` and enforced on finalize.
- Endpoints (read): `GET /api/sphinx/listings` , `GET /api/sphinx/listings/:id` , `GET /api/sphinx/listings/:id/complementary` , `GET /api/sphinx/listings/:id/syntheses` , `GET /api/sphinx/pairs/top` , `GET /api/sphinx/credits/:userId` , `GET /api/sphinx/listings/by-creator/:userId` , `GET /api/sphinx/sales/:userId` , `GET /api/sphinx/purchases/:userId` , `GET /api/sphinx/roundtable` , `GET /api/sphinx/synthesis/sessions/:id` .
- Endpoints (write): `POST /api/sphinx/hive-precheck` , `POST /api/sphinx/listings` , `DELETE /api/sphinx/listings/:id` , `POST /api/sphinx/listings/:id/purchase` , `POST /api/sphinx/listings/:id/ai-analysis` , `POST /api/sphinx/synergies/calculate` , `POST /api/sphinx/synthesis/sessions` , `POST /api/sphinx/synthesis/sessions/:id/finalize` , `POST /api/sphinx/roundtable/recompute` .
- Files: `server/sphinx.ts` , `server/synthesis.ts` , `server/zpos.ts` , `client/src/pages/marketplace.tsx` .

7.5 Matrix Forge Lab

User benefit: Drag in your `.docx` , see in seconds whether it would pass our quality gate, fix what's flagged, then send it straight to publish — no copy-paste.

- Accepts `.docx` only (strict MIME filter + 5 MB cap). Uses `mammoth` to extract raw text, bounded by an 8-second `Promise.race` timeout. Rejects files containing `vbaProject.bin` (VBA macro markers).
- Runs `runHivePrecheck()` on the extracted body and returns `{ body, fileName, precheck }` . The client renders a terminal-style streaming log (info / ok / warn / err).

- "Send to Publish" hands off to `/marketplace/publish` via a sessionStorage prefill key (`forge-lab:prefill`).
- Sub-subscriber gating: non-subscribers can preview but cannot push to publish.
- Endpoint: `POST /api/sphinx/forge-lab/run` .
- Files: `server/forgeLab.ts` , `client/src/pages/marketplace-forge-lab.tsx` .

7.6 Bonsai Seller Onboarding

User benefit: A guided 18-step walkthrough that takes you from "I have an idea" to "my first card is live and certified" — never lost, never blocked, always one click from the next move.

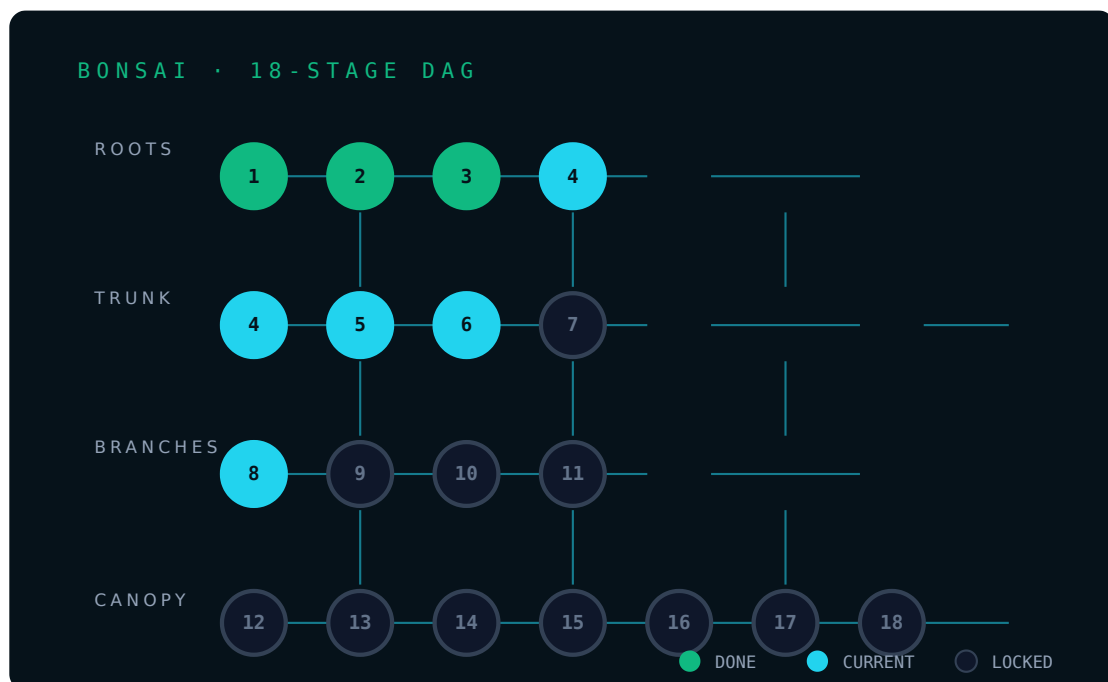


Figure 7.6 – Bonsai 18-stage DAG: Roots → Trunk → Branches → Canopy, server-enforced dependency order.

- 18 static stages declared in `shared/bonsaiStages.ts` , organized into four phases: **Roots** (1-3), **Trunk** (4-7), **Branches** (8-11), **Canopy** (12-18).
- Each stage has: title, goal, action list, declared dependencies.
- Server enforces dependency order on `POST /api/sphinx/bonsai/progress/:stageId/complete` — a stage cannot be marked complete unless all its dependencies are complete.

- **Zero ARK / HIVE side-effects** by design: Bonsai is pure pedagogy, not a flywheel input.
- Client renders the 18 stages as a `react-flow` DAG with four visual states (completed / current / locked / available), plus a detail panel with Mark-Complete and Next-Stage controls.
- Endpoints: `GET /api/sphinx/bonsai/progress` , `POST /api/sphinx/bonsai/progress/:stageId/complete` .
- Files: `shared/bonsaiStages.ts` , `server/bonsai.ts` , `client/src/pages/marketplace-bonsai.tsx` .

7.7 Cohorts & Institutional

User benefit: Run prompt-craft classes the way you'd run any course — assign work, grade it, compare cohorts, export CSV the registrar accepts.

- Instructor-gated routes via `requireInstructor` . Students see only their own cohort(s) at `GET /api/me/cohorts` .
- Bulk-add members by email at `POST /api/cohorts/:id/members` ; orphaned email rows reconcile to user IDs on the new user's first login (`reconcileCohortInvitesForUser`).
- Grade export is RFC-4180 compliant.
- Endpoints: `GET /api/cohorts` , `POST /api/cohorts` , `GET /api/cohorts/:id` , `GET /api/cohorts/comparison` , `POST /api/cohorts/:id/members` , `DELETE /api/cohorts/:id/members/:userId` , `GET /api/cohorts/:id/assignments` , `POST /api/cohorts/:id/assignments` , `GET /api/cohorts/:id/grades` , `GET /api/cohorts/:id/grades.csv` , `GET /api/me/cohorts` .
- Files: `server/routes.ts` (cohorts section), `client/src/pages/school-dashboard.tsx` .

7.8 GUIN+ Identity

User benefit: A public, citation-grade identity card — like a Stack Overflow profile crossed with an academic CV.

- Public profile at `/u/:username` rendering Knight rank, CCMI breakdown, top FORGE cards, endorsements.
- Endorsements are user-to-user; abuse-guarded by per-day rate limits and uniqueness constraints in `endorsements`.
- Endpoints: `GET /api/guin/by-id/:userId`, `GET /api/guin/by-username/:username`, `POST /api/endorsements`, `GET /api/endorsements/by-recipient/:userId`.

7.9 Billing & Subscription

User benefit: Upgrade or downgrade your plan in two clicks; receipts and history are automatic.

- Stripe-style stub (Phase D.2) with synthetic IDs. `checkoutSessions` rows track the full flow; `billing_events` is the immutable audit log.
- Subscription plans: FREE, PRO, SCHOOL_STUDENT, ENTERPRISE.
- Endpoints: `GET /api/billing/me`, `POST /api/billing/checkout`, `GET /api/billing/checkout/:id`, `POST /api/billing/checkout/:id/complete`, `POST /api/billing/cancel`, admin-only `POST /api/admin/billing/webhook-simulate`.
- Legacy `PUT /api/users/:id/subscription` is restricted to ENTERPRISE provisioning — other plans must route through `/api/billing/checkout` and receive a 409 otherwise.
- Files: `server/billing.ts`, `client/src/pages/subscription.tsx`, `client/src/pages/checkout.tsx`.

7.10 AI Surfaces (Claude)

User benefit: When you ask, the AI explains your resume in plain language, generates scenarios that match your level, and judges your prompts fairly.

- `GET /api/ai/status` exposes integration health.
- `POST /api/ai/resume-narrative/:assessmentId` — Pro-and-above users get a Sonnet-authored narrative interpretation of their JST profile.

- `POST /api/admin/ai/generate-scenario` — admin-only CCGE scenario authoring.
- All calls budgeted via `ai_usage` ledger; KCSE judgments cached in `ai_cache`.

7.11 GDPR / CCPA

User benefit: Export everything we hold on you; delete everything we hold on you. Both are one click.

- `GET /api/users/me/export` returns a full JSON dump of the user's data graph.
- `DELETE /api/users/me` with body `{ "confirm": "DELETE" }` cascades through every owned table inside a single transaction (`deleteUserCascade`).
- Files: `client/src/pages/profile.tsx` (Data Privacy section), `server/storage.ts`.

7.12 Admin & Seed

Internal benefit: Operate the platform — seed reference data, simulate webhooks, run backfills — without touching the database directly.

- `POST /api/seed` — dev-only, returns 403 in production.
- `POST /api/admin/ark/backfill` — recompute identities for all users, awarding zero positive ARK (downward sync only).
- `POST /api/admin/billing/webhook-simulate` — simulate a Stripe webhook event.
- `POST /api/admin/ccge/import-compendium` — bulk-load card sets.
- `GET /api/admin/drm/violators` + `POST /api/drm/event` — listing-body redistribution detection.

8. Phase J: ARK Identity Flywheel

Phase J is the engineering chapter that consolidated the identity surface. The contract is small, the guarantees are large.

8.1 The formulas (canonical, in `shared/schema.ts`)

```
JST = (Jobs · 0.30 + Skills · 0.40 + Talent · 0.30) · 3 // 0-300
CCMI = round(Σ pillar_score_i · weight_i) · 3 // 0-300,
P1..P7
ARK = JST + CCMI // 0-600
cap
```

The Context Craft handicap is a multiplicative modifier on the JST sub-scores **before** the formula above: `NONE 0.5×` / `CC_100 1.0×` / `CC_200 1.1×` / `CC_300 1.2×` / `CC_400 1.35×` / `CC_500 1.5×`. Raw pre-modifier values persist as `jstRawTotal` / `jstRawJobs` / `jstRawSkills` / `jstRawTalent` for audit.

8.2 The single writer

`server/arkRecalc.ts` is the only module allowed to write identity fields. Routes that look like they mutate identity (`POST /api/ark/recalc` , `POST /api/admin/ark/backfill` , every flywheel-emitting route) delegate to `recalcArkForUser` . Inside the transaction it updates four tables atomically: `users` , `ccmi_pillar_scores` , `lhcs_signals` , `ark_score_history` .

8.3 The caps and the invariant

- CCGE awards at most **+15 ARK per day per user**.
- SPHINX awards at most **+20 ARK per rolling 30 days per user**.
- `manual.recompute` and `backfill` triggers can never award positive ARK — they're downward-sync only.
- The invariant: when proportional scaling causes the JST + CCMI sum to overshoot the cap by 1 due to rounding, the overshoot is hard-clamped off the **CCMI** side, never

the JST side. This preserves the "career capital" reading of JST as the truthful number for recruiter conversations.

8.4 The LHCS composite rule

```
composite = round(0.35 · CPR + 0.35 · MPS + 0.30 · LCIS)
status    = composite ≥ 70 ? ACTIVE : composite ≥ 40 ? DEVELOPING :
BASELINE
```

The status is **not** a roll-up of the three traffic lights (CPR, MPS, LCIS individually). It is the threshold band of the composite. This was an explicit PDD decision in v3 because the prior behavior (roll-up) hid amber/red dimensions when the third dimension was strong.

8.5 The live stream

The orchestrator emits `ark.identity` on every flywheel event. The dashboard widget consumes via `useArkStream` — snapshot on mount, live merge thereafter. No polling. No localStorage. Reconnect logic is built into the EventSource hook with exponential backoff.

9. Trust, Security & Privacy

ARK ships a documented `threat_model.md`. Key guarantees:

- **Spoofing**: Server-side sessions only; production refuses to start without `SESSION_SECRET`. Bcrypt cost 10 with transparent legacy rehash. Failed-auth rate-limit at 20 / 15 min.
- **Tampering**: Every mutation derives actor from `req.session.userId`. No path or body parameter is trusted for identity. Object-ownership checks repeat inside transactions for credit / cert / score mutations.
- **Repudiation**: `billing_events`, `ark_events`, `ai_usage`, and cohort grade actions all write an immutable audit row with actor, target, timestamp, event type.
- **Information disclosure**: `GET /api/users/:id` returns full record only for self; for others a public DTO of (id, name, role, contextCraftCertLevel). `PUT /api/users/:id/profile` strips the `role` field server-side. Private SPC bodies require purchase or ownership.
- **Denial of service**: Body limits 1 MB; resume upload 10 MB; Forge Lab 5 MB. PDF parse bounded by 15 s timeout; .docx parse bounded by 8 s timeout. Global rate limit 240 / min.
- **Elevation of privilege**: `PUT /api/users/:id/context-craft-cert` returns **403 permanently** — certification level is flywheel-only. Admin endpoints require admin role; seed endpoint returns 403 in production.

10. Roadmap Delta

What shipped:

- Phase J: ARK identity consolidation, single-writer, capped flywheel, LHCS composite, SSE stream, atomic migration.
- M2: Marketplace layout + AI metadata pipeline.
- M3: Roundtable awareness + synergy social graph.
- M4: Synthesis sessions + ZPOS analyzer + complementary pairs.
- M5: Forge Lab .docx upload + HIVE pre-check; Bonsai 18-stage onboarding; ATANDA Studio branding.

What is queued (from `dev_plan.md` and `LEAN_ONECRAFT_ROADMAP.md`):

- M6: SPC versioning + creator changelog UI.
- M7: Enterprise cohort license SKUs in `billing.ts`.
- M8: External webhook intake for Stripe parity (currently stub).
- M9: GUIN+ public discovery feed.
- M10: Mobile artifact (Expo) for `/dashboard` + `/play` only.

Out of immediate scope:

- Multi-tenant white-label.
- Direct Anthropic billing pass-through (currently Replit-managed integration).
- On-premise deployment.

11. Comparative Analysis — v3 vs the Prior PDD Corpus

ARK has had eight prior PDDs across three eras. v3 is the first to absorb them into one product narrative. The matrix below compares the central axes.

11.1 Cross-PDD matrix

THEME	INTEGRATED V1 (FEB)	FORGE PDD (MAR)	JNOMICSDECK ALPHA (MAR)	ONECRAFT V2 (MAY)	ONECRAFT MVP (MAY)
Positioning thesis	JST as a career number	JST + Forge methodology	API platform for SI agents	Onecraft as governance ladder	MVP for Onecraft loop
Primary user	Job-seeker	Job-seeker + builder	API consumer	Creator	Creator
Identity formula	JST only	JST + Forge tier	n/a (agent-centric)	JST + CCMI (introduced)	JST + CCMI
Scoring surfaces	JST tri-dim	JST + JCSE	JCSE + HIVE	JST, CCMI, LHCS, JCSE, HIVE, CC_xxx	JST, CCMI, LHCS, JCSE
Marketplace model	Mentioned	Mentioned (SPHINX seed)	SPHINX as agent registry	SPHINX listings	SPHINX listings + purchase
Onboarding model	None	None	None	Implicit (CCGE)	Implicit
Governance / trust	Implicit	Implicit	None	LHCS + Knight ranks	LHCS

AI agent stance	Career AI	Career AI + prompt-craft	Agents are the product	Agents + cognitive layer	Agents + cognitive layer
Monetisation	Subscription	Subscription + cert ladder	API metering (implied)	Sub + marketplace credits	Sub + marketplace
Notable omissions vs v3	No CCMI, no marketplace	No Bonsai, no SSE	No JST, no flywheel	No Forge Lab, no Bonsai	No Forge Lab

11.2 Per-PDD review

11.2.1 Integrated v1 (Feb 2026) — **ark_pdd_integrated**

What it got right. Crisp three-question framing ("Know your worth. Know your risk. Know your next move.") that v3 still uses as the dashboard headline. Established JST as a tri-dimensional career number rather than a single percentile.

What it missed. No CCMI, no marketplace, no flywheel. Identity was static — a one-time CV upload, not a living score.

What v3 supersedes. v3 keeps the framing, replaces the static identity with the live ARK score, and adds CCMI, LHCS, the marketplace, and the flywheel.

11.2.2 ARK Forge PDD (Mar 2026) — **forge-v1**

What it got right. Introduced the Forge Promptware methodology and a JCSE 48/50 quality gate. First doc to talk about JST × cert-tier multipliers (which became the Context Craft handicap in v3).

What it missed. No SPHINX surface yet. No Bonsai. JCSE was the only quality lens — HIVE didn't exist yet.

What v3 supersedes. v3 keeps the multiplier mechanic (canonized as **CONTEXT_CRAFT_HANDICAP**), adds HIVE as the publish gate, and ships SPHINX as a real surface rather than a future promise.

11.2.3 JnomicsDeck Alpha ATLAS PDD (Mar 2026) — **jnomicsdeck-alpha**

What it got right. Defined SPHINX as a *registry* of Super Prompt Cards (SPCs) with declared ULTRA SI lineage (ADA, HOLMES, GRO / ANT KING, STRATEGOS, LUCI). Introduced HIVE as the

14-dimensional cert framework — the doc that put the publish gate on the map.

What it missed. Agent-centric framing — focused on agents-as-product, not on the *cognitive DNA layer* that governs them. No JST or user identity surface.

What v3 supersedes. v3 inverts the framing (DNA, not agents) and binds HIVE to user-side certification (CC_400 gate), turning the registry into a marketplace.

11.2.4 ARK Onecraft PDD v2 (May 2026) — onecraft-v2

What it got right. The most comprehensive prior PDD. Introduced GUIN+ identity, Knight ranks, the Onecraft compression vocabulary, and laid the groundwork for CCMI. Defined ten archetype pairs (KING / WARRIOR, ARCHITECT / SAGE, etc.) that informed the v3 archetype handicap.

What it missed. No Forge Lab. No Bonsai. ARK identity formula was implied but never canonized as the single formula.

What v3 supersedes. v3 canonizes $ARK = JST + CCMI$, ships Forge Lab + Bonsai as real surfaces, and reduces the Knight ranks to a contributor-gamification layer (decoupled from the canon Six-Stage Agent Lifecycle which v3 explicitly does *not* implement).

11.2.5 ARK Onecraft PDD MVP (May 2026) — onecraft-mvp

What it got right. First doc to commit to a shipping MVP scope: PRE_CRAFT / CRAFT_ARK / CRAFT_CCGE / POST_CRAFT loop. Introduced SAML/OIDC SSO as a stated requirement for institutions.

What it missed. SSO is still future work in v3 (institutions use local credentials). No Forge Lab. The MVP doc treated marketplace as one screen rather than a multi-surface stack.

What v3 supersedes. v3 ships the four-phase loop as Identity / Resume Analyzer / CCGE / Marketplace and adds Bonsai as the missing onboarding spine. SSO is queued explicitly in the roadmap.

11.2.6 ATANDA Command Centre MVP ATLAS PDD (May 2026) — atanda-command-centre

What it got right. Established the ATANDA brand as the parent of the product family. Introduced the SPARTAN Compression Method (SCM 7-Step) for prompt artefacts and the "VIBE DJ" orchestration concept. First PDD to commit to a single-developer, 4-week, Cursor-driven cadence.

What it missed. Operationally-framed, not user-product-framed. It does not describe what a user *sees* — it describes what an engineer *types*.

What v3 supersedes. v3 inherits ATANDA as the parent brand (sidebar footer, cover page of this document, manifesto) and treats SCM / VIBE DJ as internal engineering discipline rather than user-facing surfaces.

11.2.7 Matrix Marketplace ATLAS PDD (May 2026) — **matrix-marketplace**

What it got right. First PDD to frame the marketplace as an **asset class** — Super Prompt Cards have provenance, synergy properties, and a price discovery mechanism. Introduced the "Bonsai" metaphor for cultivating a card from seed to publish.

What it missed. No user identity surface, no JST, no CCMI. The Matrix doc is investor-facing — it answers "why this asset class" but not "why a user shows up tomorrow."

What v3 supersedes. v3 keeps the asset-class framing for SPHINX × Matrix, *and* ships the Bonsai metaphor as a real 18-stage walkthrough (not a marketing flourish), *and* binds it to the user identity loop via the CC_400 cert gate.

11.2.8 Investor ATLAS PDD (May 2026) — **investor-atlas**

What it got right. Three-part structure (cheat-sheet → executive summary → comprehensive worksheet). Color-coded prompt families (YEL-DEAL, FLYWHEEL, GRN-HIVE, PURCHASE, GRN-GUIN, BLU-BILL) that map to the actual route surface.

What it missed. Brevity. The investor doc is excellent for a five-minute pitch but cannot serve as the product specification.

What v3 supersedes. v3 keeps the executive-summary cadence at the front and the route-family taxonomy in Appendix A — but adds the depth needed to brief a new engineer or a serious due-diligence reader.

11.3 Convergence narrative

The eight prior PDDs trace a recognizable arc. **Feb 2026** asked, "What is a career number?" and answered JST. **March 2026** asked, "How do we govern the prompt-craft that surrounds that career number?" and answered Forge / JCSE / SPHINX-as-registry / HIVE. **May 2026** asked three different questions in parallel — the Onecraft pair asked "What is the user loop?", the ATANDA Command Centre asked "How do we operate this discipline?", and the Matrix / Investor pair asked "What is the asset class and how do we describe it to capital?"

v3 is the first PDD that does not pick one of those questions. It is the first PDD organized around the **product loop as a whole**: a user uploads a CV, plays the game, raises their CCMI, earns a cert, walks the Bonsai, ships their first card, sees it ranked on the Roundtable, watches a buyer compose it into a deck, and watches the ARK identity score reflect the entire journey in a single SSE-streamed number on the dashboard. That loop did not exist in

any of the prior eight documents. It exists in v3 because each prior document supplied exactly the piece v3 needed.

The manifesto — *we don't build AI agents, we engineer the DNA that governs them, powered by your cognition, owned by you* — is the one sentence that retroactively justifies every prior PDD. The Integrated v1 measured the human; Forge measured the prompt; JnomicsDeck measured the agent; Onecraft measured the loop; ATANDA measured the operator; Matrix measured the asset; Investor measured the opportunity. v3 measures the DNA that makes all six measurements meaningful.

Appendix A · Full HTTP Route Table

VERB	PATH	AUTH	PURPOSE
POST	/api/auth/register	public	Create account, auto-login
POST	/api/auth/login	public	Login, session regenerate
POST	/api/auth/logout	session	Destroy session
GET	/api/auth/me	session	Current user
GET	/api/users/:id	session	Self → full; other → public DTO
PUT	/api/users/:id/profile	requireSelf	Strips role
PUT	/api/users/:id/subscription	requireSelf	ENTERPRISE legacy only
PUT	/api/users/:id/context-craft-cert	—	403 permanent
GET	/api/context-craft/levels	public	Cert ladder reference
GET	/api/subscription/plans	public	Plan reference
POST	/api/resume/upload	session	Multipart resume
POST	/api/assessments	session	Manual assessment
POST	/api/assessment/text	session	Text-only assessment
GET	/api/assessments/user/:userId/latest	requireSelf	Latest assessment
GET	/api/assessments/user/:userId	requireSelf	Assessment history
POST	/api/notifications/assessment-summary	session	Email summary
GET	/api/notifications	session	List
POST	/api/notifications/read	session	Mark read
GET	/api/ark/identity	session	Identity payload

POST	/api/ark/recalc	session	Force recalc
GET	/api/ark/flywheel-cta	session	Next-best action
GET	/api/ark/history	session	Score trajectory
GET	/api/ark/lhcs	session	LHCS detail
GET	/api/ark-score/stream	session	SSE
GET	/api/ark-score/events	session	Fallback poll
POST	/api/admin/ark/backfill	admin	Downward-sync recalc
GET	/api/ccge/cards	session	Card catalog
GET	/api/ccge/scenarios	session	Scenario catalog
POST	/api/ccge/scenarios/custom	session	Custom scenario
GET	/api/ccge/sessions/:id	requireSelf	Single session
GET	/api/ccge/sessions/user/:userId	requireSelf	Session history
POST	/api/ccge/sessions	session	Start, deals 5
POST	/api/ccge/sessions/:id/finish	session	Atomic finalize
POST	/api/sphinx/hive-precheck	session	HIVE score preview
POST	/api/sphinx/listings	CC_400+	Publish SPC
GET	/api/sphinx/listings	public	Browse
GET	/api/sphinx/listings/:id	public	Detail
DELETE	/api/sphinx/listings/:id	owner	Remove
POST	/api/sphinx/listings/:id/purchase	session	Transactional purchase
POST	/api/sphinx/listings/:id/ai-analysis	session	Claude AI annotation
GET	/api/sphinx/listings/:id/complementary	public	Complementary pairs
GET	/api/sphinx/listings/:id/syntheses	public	Synthesis sessions
GET	/api/sphinx/credits/:userId	requireSelf	Credit balance
GET	/api/sphinx/listings/by-creator/:userId	public	Creator listings
GET	/api/sphinx/sales/:userId	requireSelf	Sales

GET	<code>/api/sphinx/purchases/:userId</code>	requireSelf	Purchases
GET	<code>/api/sphinx/pairs/top</code>	public	Top synergy pairs
GET	<code>/api/sphinx/roundtable</code>	public	Awareness graph
POST	<code>/api/sphinx/roundtable/recompute</code>	admin	Force recompute
POST	<code>/api/sphinx/synergies/calculate</code>	session	Synergy compute
POST	<code>/api/sphinx/synthesis/sessions</code>	session	Start synthesis
GET	<code>/api/sphinx/synthesis/sessions/:id</code>	session	Synthesis state
POST	<code>/api/sphinx/synthesis/sessions/:id/finalize</code>	session	Atomic finalize
POST	<code>/api/sphinx/forge-lab/run</code>	session	.docx → HIVE precheck
GET	<code>/api/sphinx/bonsai/progress</code>	session	Onboarding state
POST	<code>/api/sphinx/bonsai/progress/:stageId/complete</code>	session	Mark stage done
GET	<code>/api/guin/by-id/:userId</code>	public	GUIN+ profile
GET	<code>/api/guin/by-username/:username</code>	public	GUIN+ profile by handle
POST	<code>/api/endorsements</code>	session	Endorse another user
GET	<code>/api/endorsements/by-recipient/:userId</code>	public	Endorsement list
GET	<code>/api/cohorts</code>	instructor	Cohort list
POST	<code>/api/cohorts</code>	instructor	Create cohort
GET	<code>/api/cohorts/:id</code>	instructor	Detail
GET	<code>/api/cohorts/comparison</code>	instructor	Comparison
POST	<code>/api/cohorts/:id/members</code>	instructor	Bulk add by email
DELETE	<code>/api/cohorts/:id/members/:userId</code>	instructor	Remove member
GET	<code>/api/cohorts/:id/assignments</code>	instructor	Assignment list
POST	<code>/api/cohorts/:id/assignments</code>	instructor	Create assignment
GET	<code>/api/cohorts/:id/grades</code>	instructor	Grade view
GET	<code>/api/cohorts/:id/grades.csv</code>	instructor	Grade CSV (RFC- 4180)
GET	<code>/api/me/cohorts</code>	session	Student cohorts
GET	<code>/api/billing/me</code>	session	Billing state

POST	<code>/api/billing/checkout</code>	session	Start checkout
GET	<code>/api/billing/checkout/:id</code>	session	Checkout state
POST	<code>/api/billing/checkout/:id/complete</code>	session	Complete checkout
POST	<code>/api/billing/cancel</code>	session	Cancel subscription
POST	<code>/api/admin/billing/webhook-simulate</code>	admin	Simulate webhook
GET	<code>/api/ai/status</code>	session	Claude integration health
POST	<code>/api/ai/resume-narrative/:assessmentId</code>	Pro+	Sonnet narrative
POST	<code>/api/admin/ai/generate-scenario</code>	admin	Scenario authoring
GET	<code>/api/users/me/export</code>	session	GDPR/CCPA export
DELETE	<code>/api/users/me</code>	session	GDPR/CCPA delete
GET	<code>/api/jnomics-cards</code>	public	FORGE card list
POST	<code>/api/jnomics-cards/by-ids</code>	public	FORGE card lookup
GET	<code>/api/departments</code>	public	Departments reference
POST	<code>/api/seed</code>	dev-only	Seed reference data
GET	<code>/api/admin/drm/violators</code>	admin	DRM violators
POST	<code>/api/drm/event</code>	session	DRM event ingest
POST	<code>/api/admin/ccge/import-compendium</code>	admin	Bulk import cards

Appendix B · Score Constants & Weights

```

ARK_MAX           = 600
JST_MAX           = 300
CCMI_MAX          = 300

JST_WEIGHTS       = { jobs: 0.30, skills: 0.40, talent: 0.30 }
JST_MULTIPLIER    = 3

CCMI_PILLAR_COUNT = 7
CCMI_MULTIPLIER   = 3

KCSE_WEIGHTS      = { knowledge: 0.30, clarity: 0.30, specificity:
0.20, efficiency: 0.20 }
JCSE_TIER_THRESHOLDS = { bronze: 30, silver: 36, gold: 43, platinum: 48
}
HIVE_TIER_THRESHOLDS = { bronze: 60, silver: 70, gold: 80, platinum: 90
}
HIVE_PUBLISH_GATE  = 80           // = Gold; matches CC_400 user-cert
floor

CONTEXT_CRAFT_HANDICAP = {
  NONE:    0.50,
  CC_100:  1.00,
  CC_200:  1.10,
  CC_300:  1.20,
  CC_400:  1.35,
  CC_500:  1.50,
}

LHCS_WEIGHTS      = { CPR: 0.35, MPS: 0.35, LCIS: 0.30 }
LHCS_BANDS        = { active: 70, developing: 40 }    // composite ≥ x

FLYWHEEL_CAPS     = {
  CCGE:    { delta: +15, window: "1d" },
  SPHINX:  { delta: +20, window: "30d" },
}

ARCHETYPE_WEIGHTS = { titles: 0.40, skills: 0.35, forge_cards: 0.25 }

```

```
SPC_MIN_CERT_TO_PUBLISH = "CC_400"  
SPC_PRICE_RANGE          = [1, 5000]    // credits
```

Appendix C · Bonsai 18-Stage Map

#	PHASE	TITLE
1	Roots	Plant the Seed
2	Roots	Soil & Nutrients
3	Roots	Root System
4	Trunk	Sprout
5	Trunk	First Leaves
6	Trunk	Strengthen the Stem
7	Trunk	Prune Deadwood
8	Branches	First Branch — Constraints
9	Branches	Branch — Output Schema
10	Branches	Branch — Failure Modes
11	Branches	Canopy Spread — Pillar Fit
12	Canopy	Title & Description
13	Canopy	Pricing the Tree
14	Canopy	HIVE Pre-Check
15	Canopy	Roundtable Awareness
16	Canopy	Synergy & Pairing
17	Canopy	Publish & Watch
18	Canopy	Iterate Like a Bonsai Master

Each stage carries a goal, an action list, and a dependency set. Dependencies are enforced server-side; the client renders the eighteen as a `react-flow` DAG with completed / current / locked / available states.